

RNN_Captioning_pytorch

2026 年 6 月 10 日

1 使用循环神经网络 (RNN) 进行图像 caption 生成

在本练习中，你将实现基础的循环神经网络 (vanilla Recurrent Neural Networks)，并使用它们训练一个能够为图像生成新 caption 的模型。

```
[1]: # 配置 matplotlib 中文字体
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

# 设置单元格
import time, os, json # 导入时间、操作系统、JSON 处理相关模块
import numpy as np # 导入 NumPy 库, 用于数值计算
import torch # 导入 PyTorch 库
import matplotlib.pyplot as plt # 导入 Matplotlib 库, 用于绘图

# 从 cs231n 包中导入相关模块和函数
from cs231n.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array # 梯度检查函数
from cs231n.rnn_layers_pytorch import * # PyTorch 的 RNN 层相关实现
from cs231n.captioning_solver_pytorch import CaptioningSolverPytorch # 图像
caption 生成的求解器
from cs231n.classifiers.rnn_pytorch import CaptioningRNN # 用于 caption 生成的
RNN 分类器
from cs231n.coco_utils import load_coco_data, sample_coco_minibatch, \
    decode_captions # COCO 数据集处理工具
from cs231n.image_utils import image_from_url # 从 URL 获取图像的工具
```

```

# 设置 Matplotlib 在 notebook 中内嵌显示
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # 设置图表默认大小
plt.rcParams['image.interpolation'] = 'nearest' # 设置图像插值方式为最近邻
plt.rcParams['image.cmap'] = 'gray' # 设置默认图像颜色映射为灰度
# 加载自动重载扩展
%load_ext autoreload
# 设置自动重载模式，修改模块后自动重新导入
%autoreload 2

def rel_error(x, y):
    """ 返回相对误差 """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

```

2 COCO 数据集

在本练习中，我们将使用 [COCO 数据集](#) 的 2014 年版本，这是一个图像 caption 生成任务的标准测试集。该数据集包含 80,000 张训练图像和 40,000 张验证图像，每张图像都配有 5 条由 Amazon Mechanical Turk 平台工作者编写的 caption。

图像特征：我们已经对数据进行了预处理并提取了特征。对于所有图像，我们从在 ImageNet 上预训练的 VGG-16 网络的 fc7 层提取了特征，这些特征存储在文件 train2014_vgg16_fc7.h5 和 val2014_vgg16_fc7.h5 中。为了减少处理时间和内存需求，我们使用主成分分析（PCA）将特征维度从 4096 降至 512，这些降维后的特征存储在文件 train2014_vgg16_fc7_pca.h5 和 val2014_vgg16_fc7_pca.h5 中。原始图像占用近 20GB 空间，因此我们未包含在下载内容中。由于所有图像均来自 Flickr，我们将训练和验证图像的 URL 存储在文件 train2014_urls.txt 和 val2014_urls.txt 中，这样你可以实时下载图像用于可视化。

Captions (图像描述)：直接处理字符串效率较低，因此我们将使用编码后的 caption。每个单词都被分配一个整数 ID，这样我们就可以用整数序列表示一条 caption。整数 ID 与单词之间的映射关系在文件 coco2014_vocab.json 中，你可以使用 cs231n/coco_utils.py 文件中的 decode_captions 函数将整数 ID 的 NumPy 数组转换回字符串。

特殊标记 (Tokens)：我们在词汇表中添加了几个特殊标记，并且已经处理了所有与特殊标记相关的实现细节。我们在每条 caption 的开头添加一个特殊的 <START> 标记，在结尾添加一个 <END> 标记。罕见词会被替换为特殊的 <UNK> 标记（表示“未知”）。此外，由于我们希望在包含不同长度 caption 的小批量数据上进行训练，因此会在短 caption 的 <END> 标记后填充特殊的 <NULL> 标记，

并且不会对 <NULL> 标记计算损失或梯度。

你可以使用 `cs231n/coco_utils.py` 文件中的 `load_coco_data` 函数加载所有 COCO 数据（包括 caption、特征、URL 和词汇表）。运行下面的单元格进行加载：

```
[2]: # 将 COCO 数据从磁盘加载到一个字典中
# 在本作业的剩余部分，我们将使用降维后的特征
# 但你也可以通过修改下面的标志自行尝试原始特征
data = load_coco_data(pca_features=True) # 加载 COCO 数据，使用 PCA 降维后的特征

# 打印数据字典中的所有键和对应的值信息
for k, v in data.items():
    if type(v) == np.ndarray: # 如果值是 NumPy 数组
        print(k, type(v), v.shape, v.dtype) # 打印键、类型、形状、数据类型
    else:
        print(k, type(v), len(v)) # 打印键、类型、长度（非数组类型）
```

基础目录 `E:\学习\计算机视觉\assignment-Chinese\assignment2\cs231n\datasets/`

```
↪coco_captioning
train_captions <class 'numpy.ndarray'> (400135, 17) int32
train_image_idxs <class 'numpy.ndarray'> (400135,) int32
val_captions <class 'numpy.ndarray'> (195954, 17) int32
val_image_idxs <class 'numpy.ndarray'> (195954,) int32
train_features <class 'numpy.ndarray'> (82783, 512) float32
val_features <class 'numpy.ndarray'> (40504, 512) float32
idx_to_word <class 'list'> 1004
word_to_idx <class 'dict'> 1004
train_urls <class 'numpy.ndarray'> (82783,) <U63
val_urls <class 'numpy.ndarray'> (40504,) <U63
```

2.1 查看数据

在处理数据集之前，查看数据集中的示例总是一个好主意。

你可以使用 `cs231n/coco_utils.py` 文件中的 `sample_coco_minibatch` 函数，从 `load_coco_data` 返回的数据结构中抽取小批量数据样本。运行下面的代码，抽取一小批量训练数据，并显示图像及其对应的描述。多次运行并查看结果，有助于你了解这个数据集。

```
[62]: # 抽取一个小批量数据并显示图像及其描述
# 如果出现错误, 可能是 URL 已失效, 不用担心!
# 你可以任意多次重新抽取
batch_size = 3 # 小批量数据的大小

# 从数据中抽取小批量数据, 返回 caption、特征和图像 URL
captions, features, urls = sample_coco_minibatch(data, batch_size=batch_size)

# 遍历每个样本, 显示图像和对应的描述
for i, (caption, url) in enumerate(zip(captions, urls)):
    plt.imshow(image_from_url(url)) # 从 URL 加载并显示图像
    plt.axis('off') # 关闭坐标轴显示
    caption_str = decode_captions(caption, data['idx_to_word']) # 将整数 ID 序列解码为字符串描述
    plt.title(caption_str) # 设置图像标题为解码后的描述
    plt.show() # 显示图像
```

<START> a <UNK> chicken standing in a bunch of <UNK> surrounded by wood fence <END>



<START> a brown bear standing on top of a <UNK> <UNK> <END>



<START> a bunch of people flying kites in <UNK> <UNK> <END>



3 循环神经网络 (RNN)

正如课堂上所讨论的，我们将使用循环神经网络 (RNN) 语言模型来进行图像 caption 生成。cs231n/rnn_layers_pytorch.py 文件包含了循环神经网络所需的不同层类型的实现，而 cs231n/classifiers/rnn_pytorch.py 文件则利用这些层来实现图像 caption 生成模型。

我们首先要在 cs231n/rnn_layers_pytorch.py 中实现不同类型的 RNN 层。

4 基础 RNN：单步前向传播

打开文件 cs231n/rnn_layers_pytorch.py。该文件实现了循环神经网络中常用的不同类型层的前向传播。注意，由于我们使用 PyTorch，反向传播将由 PyTorch 的自动求导 (autograd) 机制处理。

首先实现函数 rnn_step_forward，该函数用于实现基础循环神经网络单个时间步的前向传播。完成后，运行下面的代码检查你的实现。你应该会看到误差在 $e-8$ 量级或更小。

```
[16]: N, D, H = 3, 10, 4 # N: 批量大小, D: 输入特征维度, H: 隐藏层维度

# 创建输入数据 x (形状为 N×D), 从-0.4 到 0.7 均匀生成 N×D 个值
x = torch.from_numpy(np.linspace(-0.4, 0.7, num=N*D).reshape(N, D))
# 创建前一时间步的隐藏状态 prev_h (形状为 N×H), 从-0.2 到 0.5 均匀生成 N×H 个值
prev_h = torch.from_numpy(np.linspace(-0.2, 0.5, num=N*H).reshape(N, H))
# 创建输入到隐藏层的权重矩阵 Wx (形状为 D×H), 从-0.1 到 0.9 均匀生成 D×H 个值
Wx = torch.from_numpy(np.linspace(-0.1, 0.9, num=D*H).reshape(D, H))
# 创建隐藏层到隐藏层的权重矩阵 Wh (形状为 H×H), 从-0.3 到 0.7 均匀生成 H×H 个值
Wh = torch.from_numpy(np.linspace(-0.3, 0.7, num=H*H).reshape(H, H))
# 创建偏置项 b (长度为 H), 从-0.2 到 0.4 均匀生成 H 个值
b = torch.from_numpy(np.linspace(-0.2, 0.4, num=H))

# 执行 RNN 单步前向传播, 将结果转换为 numpy 数组
next_h = rnn_step_forward(x, prev_h, Wx, Wh, b).numpy()
# 预期的输出结果 (用于验证实现正确性)
expected_next_h = np.asarray([
    [-0.58172089, -0.50182032, -0.41232771, -0.31410098],
    [ 0.66854692,  0.79562378,  0.87755553,  0.92795967],
    [ 0.97934501,  0.99144213,  0.99646691,  0.99854353]])

# 计算并打印 next_h 与预期结果的相对误差
print('next_h error: ', rel_error(expected_next_h, next_h))
```

```
next_h error:  6.292421426471037e-09
```

5 基础 RNN: 单步反向传播

由于我们使用 PyTorch 实现了 `rnn_step_forward`, 因此不需要再手动实现 `rnn_step_backward`。我们可以通过数值梯度检查器来验证 PyTorch 自动求导 (autograd) 的反向传播是否正确。

不过, 如果你有兴趣, 可以尝试自己实现 `rnn_step_backward`。但这并不是本作业的要求。

```
[17]: from cs231n.rnn_layers_pytorch import rnn_step_forward # 从对应模块导入 RNN 单步
      前向传播函数

# 创建测试输入
np.random.seed(231) # 设置随机种子, 保证结果可复现
```



```

N, D, H = 4, 5, 6 # N: 批量大小, D: 输入特征维度, H: 隐藏层维度
x = torch.from_numpy(np.random.randn(N, D)) # 输入数据 (随机正态分布)
h = torch.from_numpy(np.random.randn(N, H)) # 前一时间步隐藏状态 (随机正态分布)
Wx = torch.from_numpy(np.random.randn(D, H)) # 输入到隐藏层的权重矩阵 (随机正态分布)
Wh = torch.from_numpy(np.random.randn(H, H)) # 隐藏层到隐藏层的权重矩阵 (随机正态分布)
b = torch.from_numpy(np.random.randn(H)) # 偏置项 (随机正态分布)

# 启用梯度跟踪并执行 RNN 前向传播
for tensor in [x, h, Wx, Wh, b]:
    tensor.requires_grad_() # 为每个张量开启梯度跟踪
next_h = rnn_step_forward(x, h, Wx, Wh, b) # 计算当前时间步的隐藏状态

# 模拟随机的上游梯度, 并使用 PyTorch 的自动求导进行反向传播
dnext_h = torch.from_numpy(np.random.randn(*next_h.shape)) # 上游梯度 (形状与 next_h 相同)
next_h.backward(dnext_h) # 反向传播, 计算梯度

# 将梯度收集到单独的 numpy 数组中
dx = x.grad.detach().numpy() # x 的梯度 (转为 numpy 数组)
dh = h.grad.detach().numpy() # h 的梯度 (转为 numpy 数组)
dWx = Wx.grad.detach().numpy() # Wx 的梯度 (转为 numpy 数组)
dWh = Wh.grad.detach().numpy() # Wh 的梯度 (转为 numpy 数组)
db = b.grad.detach().numpy() # b 的梯度 (转为 numpy 数组)
dnext_h = dnext_h.detach().numpy() # 上游梯度 (转为 numpy 数组)

# 同时将测试输入转换为 numpy 数组
x = x.detach().numpy()
h = h.detach().numpy()
Wx = Wx.detach().numpy()
Wh = Wh.detach().numpy()
b = b.detach().numpy()

# 包装前向传播函数, 使其支持 numpy 数组的输入和输出
# 使用 `torch.no_grad()` 显式禁用梯度跟踪

```

```

def rnn_step_forward_numpy(x, h, Wx, Wh, b):
    with torch.no_grad():
        return rnn_step_forward(
            torch.from_numpy(x),
            torch.from_numpy(h),
            torch.from_numpy(Wx),
            torch.from_numpy(Wh),
            torch.from_numpy(b),
        ).numpy()

# 计算数值梯度并进行比较
fx = lambda x: rnn_step_forward_numpy(x, h, Wx, Wh, b) # 以 x 为变量的函数
fh = lambda h: rnn_step_forward_numpy(x, h, Wx, Wh, b) # 以 h 为变量的函数
fWx = lambda Wx: rnn_step_forward_numpy(x, h, Wx, Wh, b) # 以 Wx 为变量的函数
fWh = lambda Wh: rnn_step_forward_numpy(x, h, Wx, Wh, b) # 以 Wh 为变量的函数
fb = lambda b: rnn_step_forward_numpy(x, h, Wx, Wh, b) # 以 b 为变量的函数

# 计算各参数的数值梯度
dx_num = eval_numerical_gradient_array(fx, x, dnext_h)
dh_num = eval_numerical_gradient_array(fh, h, dnext_h)
dWx_num = eval_numerical_gradient_array(fWx, Wx, dnext_h)
dWh_num = eval_numerical_gradient_array(fWh, Wh, dnext_h)
db_num = eval_numerical_gradient_array(fb, b, dnext_h)

# 你应该会看到误差在 1e-9 量级或更小
print('dx error: ', rel_error(dx_num, dx)) # 打印 dx 的数值梯度与自动梯度的相对误差
print('dh error: ', rel_error(dh_num, dh)) # 打印 dh 的数值梯度与自动梯度的相对误差
print('dWx error: ', rel_error(dWx_num, dWx)) # 打印 dWx 的数值梯度与自动梯度的相对误差
print('dWh error: ', rel_error(dWh_num, dWh)) # 打印 dWh 的数值梯度与自动梯度的相对误差
print('db error: ', rel_error(db_num, db)) # 打印 db 的数值梯度与自动梯度的相对误差

```

dx error: 2.319932372313319e-10

```
dh error: 2.6828355645784327e-10
dWx error: 8.820244454238703e-10
dWh error: 4.703287554560559e-10
db error: 1.5956895526227225e-11
```

6 基础 RNN：完整前向传播

既然你已经实现了基础 RNN 单个时间步的前向传播，接下来你将使用它来实现一个能够处理整个数据序列的 RNN。

在文件 `cs231n/rnn_layers_pytorch.py` 中，实现函数 `rnn_forward`。该函数应使用你上面定义的 `rnn_step_forward` 函数来实现。完成后，运行下面的代码检查你的实现。你应该会看到误差在 $e-7$ 量级或更小。

```
[18]: from cs231n.rnn_layers_pytorch import rnn_forward # 从对应模块导入 RNN 完整前向传播函数

# 定义参数:  $N$  (批量大小)、 $T$  (时间步数)、 $D$  (输入特征维度)、 $H$  (隐藏层维度)
N, T, D, H = 2, 3, 4, 5

# 创建输入数据  $x$  (形状为  $N \times T \times D$ ), 从  $-0.1$  到  $0.3$  均匀生成  $N \times T \times D$  个值
x = torch.from_numpy(np.linspace(-0.1, 0.3, num=N*T*D).reshape(N, T, D))
# 创建初始隐藏状态  $h_0$  (形状为  $N \times H$ ), 从  $-0.3$  到  $0.1$  均匀生成  $N \times H$  个值
h0 = torch.from_numpy(np.linspace(-0.3, 0.1, num=N*H).reshape(N, H))
# 创建输入到隐藏层的权重矩阵  $W_x$  (形状为  $D \times H$ ), 从  $-0.2$  到  $0.4$  均匀生成  $D \times H$  个值
Wx = torch.from_numpy(np.linspace(-0.2, 0.4, num=D*H).reshape(D, H))
# 创建隐藏层到隐藏层的权重矩阵  $W_h$  (形状为  $H \times H$ ), 从  $-0.4$  到  $0.1$  均匀生成  $H \times H$  个值
Wh = torch.from_numpy(np.linspace(-0.4, 0.1, num=H*H).reshape(H, H))
# 创建偏置项  $b$  (长度为  $H$ ), 从  $-0.7$  到  $0.1$  均匀生成  $H$  个值
b = torch.from_numpy(np.linspace(-0.7, 0.1, num=H))

# 执行 RNN 完整前向传播, 获取所有时间步的隐藏状态, 转换为 numpy 数组
h = rnn_forward(x, h0, Wx, Wh, b).numpy()
# 预期的输出结果 (用于验证实现正确性)
expected_h = np.asarray([
    [
        [-0.42070749, -0.27279261, -0.11074945, 0.05740409, 0.22236251],
```

```

        [-0.39525808, -0.22554661, -0.0409454, 0.14649412, 0.32397316],
        [-0.42305111, -0.24223728, -0.04287027, 0.15997045, 0.35014525],
    ],
    [
        [-0.55857474, -0.39065825, -0.19198182, 0.02378408, 0.23735671],
        [-0.27150199, -0.07088804, 0.13562939, 0.33099728, 0.50158768],
        [-0.51014825, -0.30524429, -0.06755202, 0.17806392, 0.40333043]]])

# 计算并打印 h 与预期结果的相对误差
print('h error: ', rel_error(expected_h, h))

```

h error: 7.728466180186066e-08

7 基础 RNN：反向传播

和之前一样，我们可以使用数值梯度检查器来验证 PyTorch 自动求导的反向传播是否正确。如果你愿意，也可以尝试自己实现 `rnn_step_backward`。但这并不是本作业的要求。

```

[19]: from cs231n.rnn_layers_pytorch import rnn_forward # 从对应模块导入 RNN 完整前向传播函数

# 创建测试输入
np.random.seed(231) # 设置随机种子，保证结果可复现
# N: 批量大小, D: 输入特征维度, T: 时间步数, H: 隐藏层维度
N, D, T, H = 2, 3, 10, 5
x = torch.from_numpy(np.random.randn(N, T, D)) # 输入数据（随机正态分布，形状为 N×T×D）
h0 = torch.from_numpy(np.random.randn(N, H)) # 初始隐藏状态（随机正态分布，形状为 N×H）
Wx = torch.from_numpy(np.random.randn(D, H)) # 输入到隐藏层的权重矩阵（随机正态分布，形状为 D×H）
Wh = torch.from_numpy(np.random.randn(H, H)) # 隐藏层到隐藏层的权重矩阵（随机正态分布，形状为 H×H）
b = torch.from_numpy(np.random.randn(H)) # 偏置项（随机正态分布，长度为 H）

# 启用梯度跟踪并执行前向传播
for tensor in [x, h0, Wx, Wh, b]:

```

```

    tensor.requires_grad_() # 为每个张量开启梯度跟踪
h = rnn_forward(x, h0, Wx, Wh, b) # 计算所有时间步的隐藏状态

# 模拟随机的上游梯度, 并使用 PyTorch 的自动求导进行反向传播
dh = torch.from_numpy(np.random.randn(*h.shape)) # 上游梯度 (形状与 h 相同)
h.backward(dh) # 反向传播, 计算梯度

# 将梯度收集到单独的 numpy 数组中
dx = x.grad.detach().numpy() # x 的梯度 (转为 numpy 数组)
dh0 = h0.grad.detach().numpy() # h0 的梯度 (转为 numpy 数组)
dWx = Wx.grad.detach().numpy() # Wx 的梯度 (转为 numpy 数组)
dWh = Wh.grad.detach().numpy() # Wh 的梯度 (转为 numpy 数组)
db = b.grad.detach().numpy() # b 的梯度 (转为 numpy 数组)
dh = dh.detach().numpy() # 上游梯度 (转为 numpy 数组)

# 同时将测试输入转换为 numpy 数组
x = x.detach().numpy()
h0 = h0.detach().numpy()
Wx = Wx.detach().numpy()
Wh = Wh.detach().numpy()
b = b.detach().numpy()

# 包装前向传播函数, 使其支持 numpy 数组的输入和输出
# 使用 `torch.no_grad()` 显式禁用梯度跟踪
def rnn_forward_numpy(x, h0, Wx, Wh, b):
    with torch.no_grad():
        return rnn_forward(
            torch.from_numpy(x),
            torch.from_numpy(h0),
            torch.from_numpy(Wx),
            torch.from_numpy(Wh),
            torch.from_numpy(b),
        ).numpy()

# 定义以不同参数为变量的函数, 用于数值梯度计算
fx = lambda x: rnn_forward_numpy(x, h0, Wx, Wh, b) # 以 x 为变量的函数

```



```

fh0 = lambda h0: rnn_forward_numpy(x, h0, Wx, Wh, b) # 以  $h_0$  为变量的函数
fWx = lambda Wx: rnn_forward_numpy(x, h0, Wx, Wh, b) # 以  $Wx$  为变量的函数
fWh = lambda Wh: rnn_forward_numpy(x, h0, Wx, Wh, b) # 以  $Wh$  为变量的函数
fb = lambda b: rnn_forward_numpy(x, h0, Wx, Wh, b) # 以  $b$  为变量的函数

# 计算各参数的数值梯度
dx_num = eval_numerical_gradient_array(fx, x, dh)
dh0_num = eval_numerical_gradient_array(fh0, h0, dh)
dWx_num = eval_numerical_gradient_array(fWx, Wx, dh)
dWh_num = eval_numerical_gradient_array(fWh, Wh, dh)
db_num = eval_numerical_gradient_array(fb, b, dh)

# 你应该会看到误差在  $1e-6$  量级或更小
print('dx error: ', rel_error(dx_num, dx)) # 打印  $dx$  的数值梯度与自动梯度的相对误差
print('dh0 error: ', rel_error(dh0_num, dh0)) # 打印  $dh_0$  的数值梯度与自动梯度的相对误差
print('dWx error: ', rel_error(dWx_num, dWx)) # 打印  $dWx$  的数值梯度与自动梯度的相对误差
print('dWh error: ', rel_error(dWh_num, dWh)) # 打印  $dWh$  的数值梯度与自动梯度的相对误差
print('db error: ', rel_error(db_num, db)) # 打印  $db$  的数值梯度与自动梯度的相对误差

```

```

dx error:  2.4180851680932135e-09
dh0 error: 3.3811400640982614e-09
dWx error: 7.2217238007937085e-09
dWh error: 1.2821246475483426e-07
db error:  4.676348457243789e-10

```

8 词嵌入：前向传播

在深度学习系统中，我们通常使用向量来表示单词。词汇表中的每个单词都会与一个向量相关联，这些向量会与系统的其他部分一起被联合学习。

在文件 `cs231n/rnn_layers_pytorch.py` 中，实现函数 `word_embedding_forward`，将单词（以整数表示）转换为向量。运行下面的代码检查你的实现。你应该会看到误差在 $e-8$ 量级或更小。

```
[20]: # 定义参数:  $N$  (批量大小)、 $T$  (时间步数/序列长度)、 $V$  (词汇表大小)、 $D$  (词向量维度)
N, T, V, D = 2, 4, 5, 3
```

```
# 创建输入  $x$  (形状为  $N \times T$ ), 每个元素是词汇表中的整数索引
```

```
x = torch.from_numpy(np.asarray([[0, 3, 1, 2], [2, 1, 0, 3]]))
```

```
# 创建词嵌入矩阵  $W$  (形状为  $V \times D$ ), 从 0 到 1 均匀生成  $V \times D$  个值
```

```
W = torch.from_numpy(np.linspace(0, 1, num=V*D).reshape(V, D))
```

```
# 执行词嵌入前向传播, 将整数索引转换为词向量, 结果转为 numpy 数组
```

```
out = word_embedding_forward(x, W).numpy()
```

```
# 预期的输出结果 (用于验证实现正确性)
```

```
expected_out = np.asarray([
    [[ 0.,          0.07142857,  0.14285714],
     [ 0.64285714,  0.71428571,  0.78571429],
     [ 0.21428571,  0.28571429,  0.35714286],
     [ 0.42857143,  0.5,          0.57142857]],
    [[ 0.42857143,  0.5,          0.57142857],
     [ 0.21428571,  0.28571429,  0.35714286],
     [ 0.,          0.07142857,  0.14285714],
     [ 0.64285714,  0.71428571,  0.78571429]]])
```

```
# 计算并打印 out 与预期结果的相对误差
```

```
print('out error: ', rel_error(expected_out, out))
```

```
out error:  1.0000000094736443e-08
```

9 词嵌入: 反向传播

和之前一样, 我们可以使用数值梯度检查器来验证 PyTorch 自动求导的反向传播是否正确。如果你愿意, 也可以尝试自己实现 `word_embedding_backward`。但这并不是本作业的要求。

```
[21]: np.random.seed(231) # 设置随机种子, 确保结果可复现
```

```
# 定义参数:  $N$  (批量大小)、 $T$  (时间步数/序列长度)、 $V$  (词汇表大小)、 $D$  (词向量维度)
```

```
N, T, V, D = 50, 3, 5, 6
```

```
# 生成输入  $x$  (形状为  $N \times T$ ), 元素为 0 到  $V-1$  之间的随机整数 (表示单词索引)
```

```
x = torch.from_numpy(np.random.randint(V, size=(N, T)))
```

```

# 生成词嵌入矩阵  $W$  (形状为  $V \times D$ ), 元素为随机正态分布值
W = torch.from_numpy(np.random.randn(V, D))
W.requires_grad_() # 开启  $W$  的梯度跟踪

# 执行词嵌入前向传播, 得到输出  $out$ 
out = word_embedding_forward(x, W)
# 生成随机的上游梯度  $dout$  (形状与  $out$  相同)
dout = torch.from_numpy(np.random.randn(*out.shape))
out.backward(dout) # 反向传播, 计算梯度

# 提取  $W$  的梯度并转换为 numpy 数组
dW = W.grad.detach().numpy()
# 将输入  $x$ 、词嵌入矩阵  $W$  和上游梯度  $dout$  转换为 numpy 数组
x = x.detach().numpy()
W = W.detach().numpy()
dout = dout.detach().numpy()

# 包装词嵌入前向传播函数, 使其支持 numpy 数组输入和输出
# 禁用梯度跟踪以提高效率
def word_embedding_forward_numpy(x, W):
    return word_embedding_forward(
        torch.from_numpy(x),
        torch.from_numpy(W),
    ).numpy()

# 定义以  $W$  为变量的函数, 用于数值梯度计算
f = lambda W: word_embedding_forward_numpy(x, W)
# 计算  $W$  的数值梯度
dW_num = eval_numerical_gradient_array(f, W, dout)

# 你应该会看到误差在  $1e-11$  量级或更小
print('dW error: ', rel_error(dW, dW_num)) # 打印  $dW$  的自动梯度与数值梯度的相对误差

```

dW error: 3.2774595693100364e-12

10 时序仿射层

在每个时间步，我们使用仿射函数将该时间步的 RNN 隐藏向量转换为词汇表中每个单词的得分。由于这与你在第二次作业中实现的仿射层非常相似，我们已经在 `temporal_affine_forward` 中为你提供了这个函数。运行下面的代码对实现进行数值梯度检查。你应该会看到误差在 $e-9$ 量级或更小。

```
[22]: np.random.seed(231) # 设置随机种子，确保结果可复现

# 对时序仿射层进行梯度检查
# N: 批量大小, T: 时间步数, D: 输入特征维度, M: 输出特征维度 (词汇表大小)
N, T, D, M = 2, 3, 4, 5
# 生成输入 x (形状为 N×T×D), 元素为随机正态分布值
x = torch.from_numpy(np.random.randn(N, T, D))
# 生成权重矩阵 w (形状为 D×M), 元素为随机正态分布值
w = torch.from_numpy(np.random.randn(D, M))
# 生成偏置项 b (长度为 M), 元素为随机正态分布值
b = torch.from_numpy(np.random.randn(M))

# 为输入、权重和偏置开启梯度跟踪
for tensor in [x, w, b]:
    tensor.requires_grad_()
# 执行时序仿射层前向传播
out = temporal_affine_forward(x, w, b)
# 生成随机的上游梯度 dout (形状与 out 相同)
dout = torch.from_numpy(np.random.randn(*out.shape))
# 反向传播, 计算梯度
out.backward(dout)

# 提取各参数的梯度并转换为 numpy 数组
dx = x.grad.detach().numpy()
dw = w.grad.detach().numpy()
db = b.grad.detach().numpy()

# 将输入、权重、偏置和上游梯度转换为 numpy 数组
x = x.detach().numpy()
w = w.detach().numpy()
```

```

b = b.detach().numpy()
dout = dout.detach().numpy()

# 包装时序仿射层前向传播函数, 使其支持 numpy 数组输入和输出
def temporal_affine_forward_numpy(x, w, b):
    return temporal_affine_forward(
        torch.from_numpy(x),
        torch.from_numpy(w),
        torch.from_numpy(b),
    ).numpy()

# 定义以不同参数为变量的函数, 用于数值梯度计算
fx = lambda x: temporal_affine_forward_numpy(x, w, b) # 以 x 为变量的函数
fw = lambda w: temporal_affine_forward_numpy(x, w, b) # 以 w 为变量的函数
fb = lambda b: temporal_affine_forward_numpy(x, w, b) # 以 b 为变量的函数

# 计算各参数的数值梯度
dx_num = eval_numerical_gradient_array(fx, x, dout)
dw_num = eval_numerical_gradient_array(fw, w, dout)
db_num = eval_numerical_gradient_array(fb, b, dout)

# 打印各参数的自动梯度与数值梯度的相对误差
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))

```

```

dx error:  2.9215854231394017e-10
dw error:  1.5772088618663602e-10
db error:  3.252209560097257e-11

```

11 时序 softmax 损失

在 RNN 语言模型中, 每个时间步我们都会为词汇表中的每个单词生成一个得分。我们知道每个时间步的真实单词, 因此我们使用 softmax 损失函数来计算每个时间步的损失和梯度。我们对所有时间步的损失求和, 然后在小批量数据上取平均值。

但这里有一个需要注意的地方: 由于我们处理的是小批量数据, 且不同的描述可能有不同的长度, 因此我们会在每个描述的末尾添加 `<NULL>` 标记, 使它们具有相同的长度。我们不希望这些 `<NULL>`

标记对损失或梯度产生影响，因此除了得分和真实标签外，我们的损失函数还接受一个 `mask` 数组，用于指示哪些得分元素需要计入损失。

由于这与你在第一次作业中实现的 `softmax` 损失函数非常相似，我们已经为你实现了这个损失函数；你可以查看 `cs231n/rnn_layers_pytorch.py` 文件中的 `temporal_softmax_loss` 函数。

运行下面的单元格，对损失进行合理性检查，并对该函数进行数值梯度检查。你应该会看到 `dx` 的误差在 e^{-7} 量级或更小。

```
[26]: # 对时序 softmax 损失进行合理性检查
from cs231n.rnn_layers_pytorch import temporal_softmax_loss # 从对应模块导入时序
softmax 损失函数

N, T, V = 100, 1, 10 # N: 批量大小, T: 时间步数, V: 词汇表大小

def check_loss(N, T, V, p):
    # 生成输入 x (形状为 N×T×V), 值很小 (接近 0), 减少数值误差影响
    x = 0.001 * torch.from_numpy(np.random.randn(N, T, V))
    # 生成真实标签 y (形状为 N×T), 元素为 0 到 V-1 之间的随机整数
    y = torch.from_numpy(np.random.randint(V, size=(N, T)))
    # 生成 mask 数组 (形状为 N×T), 元素为布尔值, True 的概率为 p (表示该位置计入损失)
    mask = torch.from_numpy(np.random.rand(N, T) <= p)
    # 计算并打印时序 softmax 损失值
    print(temporal_softmax_loss(x, y, mask).item())

# 测试不同参数下的损失值 (用于合理性检查)
check_loss(100, 1, 10, 1.0) # 应约为 2.3 (ln(10) 2.3)
check_loss(100, 10, 10, 1.0) # 应约为 23 (10 个时间步, 损失累加)
check_loss(5000, 10, 10, 0.1) # 应在 2.2-2.4 之间 (mask 筛选后约 10% 的元素有效)

# 对时序 softmax 损失进行梯度检查
np.random.seed(231231) # 设置随机种子, 确保结果可复现
N, T, V = 7, 8, 9 # 重新定义参数

# 生成输入 x (形状为 N×T×V), 元素为随机正态分布值
x = torch.from_numpy(np.random.randn(N, T, V))
# 生成真实标签 y (形状为 N×T)
```



```

y = torch.from_numpy(np.random.randint(V, size=(N, T)))
# 生成 mask 数组 (形状为 N×T), 元素为布尔值 (随机筛选约 50% 的元素有效)
mask = torch.from_numpy(np.random.rand(N, T) > 0.5)

x.requires_grad_() # 开启 x 的梯度跟踪
# 计算时序 softmax 损失 (关闭详细输出)
loss = temporal_softmax_loss(x, y, mask, verbose=False)
loss.backward() # 反向传播, 计算梯度
dx = x.grad.detach().numpy() # 提取 x 的梯度并转换为 numpy 数组
x = x.detach().numpy() # 将 x 转换为 numpy 数组

# 计算 x 的数值梯度
dx_num = eval_numerical_gradient(
    lambda x: temporal_softmax_loss(torch.from_numpy(x), y, mask), x,
    verbose=False)

# 打印 x 的自动梯度与数值梯度的相对误差
print('dx error: ', rel_error(dx, dx_num))

```

```

2.3025096326515144
23.025995103619735
2.270804504459016
dx error: 5.5228696963589426e-08

```

12 用于图像描述生成的 RNN

既然你已经实现了所需的各个层, 现在可以将它们组合起来构建一个图像描述生成模型。打开文件 `cs231n/classifiers/rnn_pytorch.py` 并查看 `CaptioningRNN` 类。

在 `loss` 函数中实现模型的前向传播。目前你只需要实现 `cell_type='rnn'` (基础 RNN) 的情况; 稍后你将实现 LSTM 的情况。完成后, 运行下面的代码, 使用一个小的测试用例检查你的前向传播; 你应该会看到误差在 $e-10$ 量级或更小。

```

[38]: # 定义参数: N (批量大小)、D (图像特征维度)、W (词向量维度)、H (隐藏层维度)
N, D, W, H = 10, 20, 30, 40
# 单词到索引的映射字典 (<NULL> 为填充标记, cat 和 dog 为示例单词)
word_to_idx = {'<NULL>': 0, 'cat': 2, 'dog': 3}
V = len(word_to_idx) # 词汇表大小 (等于映射字典的长度)

```

```

T = 13 # 每个描述的时间步数 (序列长度)

# 创建图像描述生成模型 (RNN)
model = CaptioningRNN(
    word_to_idx,      # 单词到索引的映射
    input_dim=D,      # 输入图像特征的维度
    wordvec_dim=W,     # 词向量的维度
    hidden_dim=H,     # 隐藏层的维度
    cell_type='rnn',   # 使用基础 RNN 单元
    dtype=torch.float64 # 数据类型 (双精度浮点数, 减少数值误差)
)

# 将模型的所有参数设置为固定值 (用于测试一致性)
for k, v in model.params.items():
    # 生成从 -1.4 到 1.3 均匀分布的数值, 形状与参数 v 一致
    model.params[k] = torch.from_numpy(
        np.linspace(-1.4, 1.3, num=v.numel()).reshape(*v.shape))

# 生成输入图像特征 (形状为  $N \times D$ ), 值从 -1.5 到 0.3 均匀分布
features = torch.from_numpy(np.linspace(-1.5, 0.3, num=(N * D)).reshape(N, D))
# 生成描述序列 (形状为  $N \times T$ ), 元素为 0 到  $V-1$  的循环整数 (模拟单词索引)
captions = torch.from_numpy((np.arange(N * T) % V).reshape(N, T))

# 计算模型的损失值 (标量)
loss = model.loss(features, captions).item()
# 预期的损失值 (用于验证实现正确性)
expected_loss = 9.83235591003

# 打印计算得到的损失、预期损失以及两者的差值
print('loss: ', loss)
print('expected loss: ', expected_loss)
print('difference: ', abs(loss - expected_loss))

```

```

loss: 9.83235591002739
expected loss: 9.83235591003
difference: 2.609468197078968e-12

```

运行下面的单元格，对 CaptioningRNN 类进行数值梯度检查；你应该会看到误差在 $e-6$ 量级左右或更小。

```
[39]: np.random.seed(231) # 设置 numpy 随机种子，确保结果可复现
      torch.manual_seed(231) # 设置 PyTorch 随机种子，确保结果可复现

# 定义参数：批量大小、时间步数、输入维度、词向量维度、隐藏层维度
batch_size = 2
timesteps = 3
input_dim = 4
wordvec_dim = 5
hidden_dim = 6
# 单词到索引的映射字典
word_to_idx = {'<NULL>': 0, 'cat': 2, 'dog': 3}
vocab_size = len(word_to_idx) # 词汇表大小

# 生成描述序列（形状为 batch_size x timesteps），元素为随机词汇索引
captions = torch.from_numpy(np.random.randint(vocab_size, size=(batch_size,
    timesteps)))

# 生成图像特征（形状为 batch_size x input_dim），元素为随机正态分布值
features = torch.from_numpy(np.random.randn(batch_size, input_dim))

# 创建图像描述生成模型（RNN）
model = CaptioningRNN(
    word_to_idx, # 单词到索引的映射
    input_dim=input_dim, # 输入图像特征的维度
    wordvec_dim=wordvec_dim, # 词向量的维度
    hidden_dim=hidden_dim, # 隐藏层的维度
    cell_type='rnn', # 使用基础 RNN 单元
    dtype=torch.float64, # 数据类型（双精度浮点数）
)

# 为模型所有参数开启梯度跟踪
for k, v in model.params.items():
    v.requires_grad_()

# 计算损失
loss = model.loss(features, captions)
```

```

# 反向传播计算梯度
loss.backward()
# 收集各参数的梯度并转换为 numpy 数组
grads = {k: v.grad.detach().numpy() for k, v in model.params.items()}
# 关闭所有参数的梯度跟踪
for k, v in model.params.items():
    v.requires_grad_(False)

# 对每个参数进行数值梯度检查
for param_name in sorted(grads.keys()):
    # 定义一个函数, 用于计算当参数为 val 时的损失
    def fn(val):
        model.params[param_name] = torch.from_numpy(val) # 将 val 设置为当前参数值
        ret = model.loss(features, captions).numpy() # 计算损失并转为 numpy 数组
        return ret

    # 计算该参数的数值梯度
    param_grad_num = eval_numerical_gradient(
        fn, model.params[param_name].numpy(), verbose=False, h=1e-6)

    # 计算数值梯度与自动梯度的相对误差
    e = rel_error(param_grad_num, grads[param_name])
    # 打印参数名及其相对误差
    print('%s relative error: %e' % (param_name, e))

```

```

W_embed relative error: 7.405825e-09
W_proj relative error: 1.627846e-08
W_vocab relative error: 5.813051e-09
W_h relative error: 3.722645e-09
W_x relative error: 3.272434e-06
b relative error: 2.259961e-10
b_proj relative error: 7.720396e-10
b_vocab relative error: 4.089136e-09

```

13 在小数据集上过度拟合 RNN 图像描述模型

与我们在之前作业中用于训练图像分类模型的 Solver 类类似，在本作业中，我们使用 CaptioningSolverPytorch 类来训练图像描述模型。打开文件 cs231n/captioning_solver_pytorch.py，阅读 CaptioningSolverPytorch 类的代码；它看起来会非常熟悉。

在熟悉了这个 API 之后，运行下面的代码，确保你的模型能够在包含 100 个训练样本的小数据集上过度拟合。你应该会看到最终损失小于 0.1。

```
[40]: np.random.seed(231) # 设置 numpy 随机种子，保证结果可复现
      torch.manual_seed(231) # 设置 PyTorch 随机种子，保证结果可复现

      # 加载 COCO 数据集的小样本（最多 50 个训练样本）
      small_data = load_coco_data(max_train=50)

      # 创建一个基于 RNN 的图像描述模型
      small_rnn_model = CaptioningRNN(
          cell_type='rnn', # 使用基础 RNN 单元
          word_to_idx=data['word_to_idx'], # 单词到索引的映射字典
          input_dim=data['train_features'].shape[1], # 输入图像特征的维度
          hidden_dim=512, # 隐藏层维度
          wordvec_dim=256, # 词向量维度
      )

      # 创建图像描述模型的求解器（用于训练）
      small_rnn_solver = CaptioningSolverPytorch(
          small_rnn_model, small_data, # 模型和数据集
          num_epochs=50, # 训练的轮数
          batch_size=25, # 批量大小
          learning_rate=5e-3, # 学习率
          verbose=True, # 训练过程中打印详细信息
          print_every=10, # 每 10 次迭代打印一次信息
      )

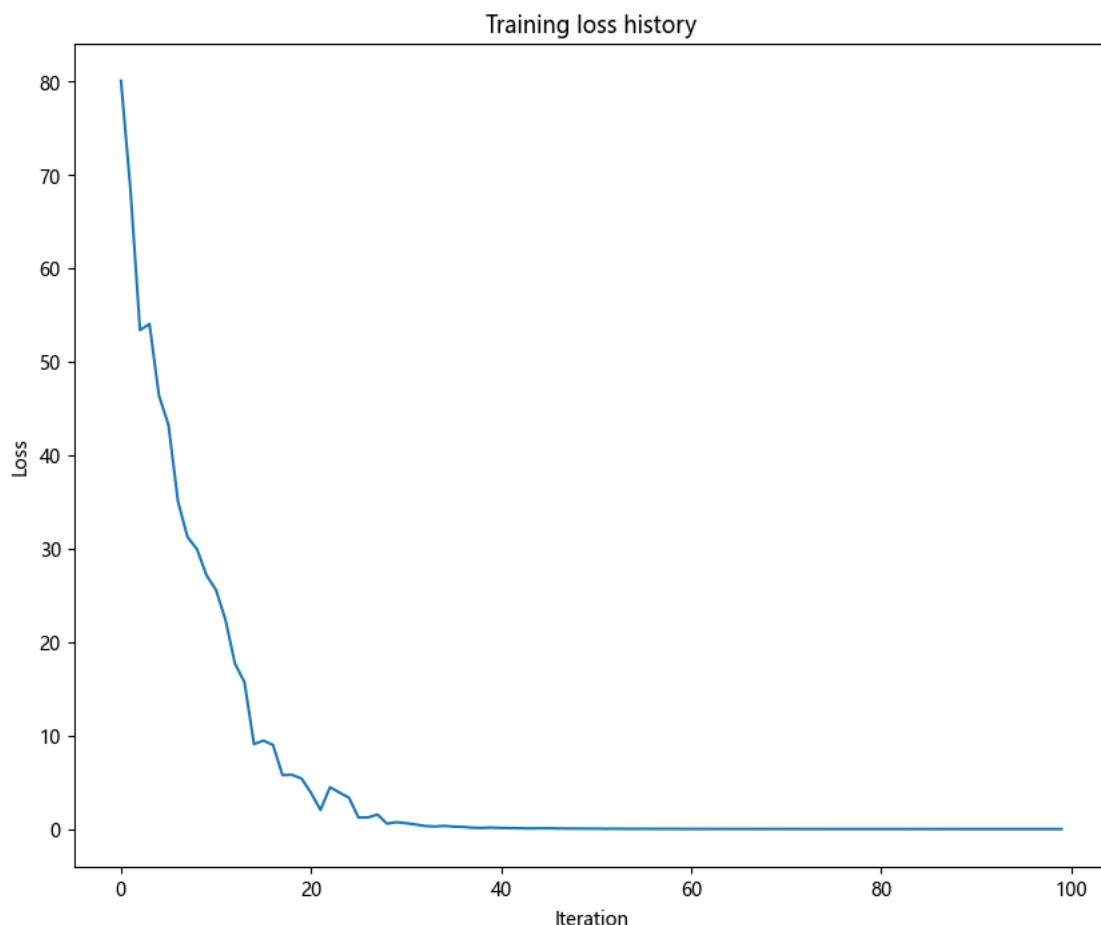
      # 开始训练模型
      small_rnn_solver.train()
```

```
# 绘制训练损失曲线
plt.plot(small_rnn_solver.loss_history)
plt.xlabel('Iteration') # x 轴标签: 迭代次数
plt.ylabel('Loss') # y 轴标签: 损失值
plt.title('Training loss history') # 图表标题: 训练损失历史
plt.show() # 显示图表
```

基础目录 E:\学习\计算机视觉\assignment-Chinese\assignment2\cs231n\datasets/

↳ coco_captioning

(迭代 1 / 100) 损失: 80.027161
(迭代 11 / 100) 损失: 25.581087
(迭代 21 / 100) 损失: 3.890298
(迭代 31 / 100) 损失: 0.641083
(迭代 41 / 100) 损失: 0.133302
(迭代 51 / 100) 损失: 0.057616
(迭代 61 / 100) 损失: 0.028606
(迭代 71 / 100) 损失: 0.022349
(迭代 81 / 100) 损失: 0.018518
(迭代 91 / 100) 损失: 0.016730



打印最终的训练损失。你应该会看到最终损失小于 0.1。

```
[41]: # 打印训练过程中的最终损失值
# loss_history 存储了训练过程中每一步的损失, [-1] 表示取最后一个元素 (即最终损失)
print('Final loss: ', small_rnn_solver.loss_history[-1])
```

```
Final loss: 0.01337142
```

14 RNN 在测试时的采样

与分类模型不同，图像描述模型在训练时和测试时的行为有很大差异。在训练时，我们可以获取真实的描述文本，因此在每个时间步，我们将真实单词作为输入馈送到 RNN 中。而在测试时，我们在每个时间步从词汇表的概率分布中进行采样，并将该采样结果作为下一个时间步的输入馈送到 RNN 中。

在文件 `cs231n/classifiers/rnn_pytorch.py` 中, 实现用于测试时采样的 `sample` 方法。完成后, 运行下面的代码, 从你的过拟合模型中对训练数据和验证数据进行采样。训练数据上的采样结果应该会很好。不过, 验证数据上的采样结果可能没什么意义。

```
[60]: # 如果你遇到错误, 可能是 URL 已失效, 不用担心!
# 你可以根据需要多次重新采样

# 分别对训练集和验证集进行采样测试
for split in ['train', 'val']:
    # 从数据集中抽取一个小批量样本 (包含真实描述、图像特征和图像 URL)
    minibatch = sample_coco_minibatch(small_data, split=split, batch_size=2)
    gt_captions, features, urls = minibatch
    # 将真实描述的索引转换为文字 (解码)
    gt_captions = decode_captions(gt_captions, data['idx_to_word'])

    # 使用训练好的模型对图像特征进行采样, 生成描述 (索引形式), 再转换为 numpy 数组
    sample_captions = small_rnn_model.sample(torch.from_numpy(features)).numpy()
    # 将生成的描述索引转换为文字 (解码)
    sample_captions = decode_captions(sample_captions, data['idx_to_word'])

    # 遍历每个样本, 显示图像及对应的真实描述和生成描述
    for gt_caption, sample_caption, url in zip(gt_captions, sample_captions, u
↪ urls):
        # 从 URL 加载图像
        img = image_from_url(url)
        # 跳过无效的 URL (图像无法加载的情况)
        if img is None: continue
        # 显示图像
        plt.imshow(img)
        # 设置图像标题, 包含数据集划分 (训练/验证)、生成的描述和真实描述
        plt.title('%s\n%s\nGT:%s' % (split, sample_caption, gt_caption))
        # 关闭坐标轴显示
        plt.axis('off')
        # 显示图像
        plt.show()
```

URL 错误: Not Found http://farm7.staticflickr.com/6040/6314343187_10d3a6c297_z.jpg

train

a surfer rides a large wave while the sun <UNK> the <UNK> <END>

GT:<START> a surfer rides a large wave while the sun <UNK> the <UNK> <END>



val
train tracks <END>
GT:<START> a <UNK> blue motorcycle parked near the road <END>



val
boat <END>
GT:<START> a table has a hot dog and french fries on it <END>



15 内联问题 1

在我们当前的图像描述设置中，我们的 RNN 语言模型在每个时间步都会输出一个单词作为其结果。然而，另一种解决该问题的方式是训练网络以**字符**（例如‘a’、‘b’等）为操作单位，而不是单词，这样在每个时间步，网络会接收前一个字符作为输入，并尝试预测序列中的下一个字符。例如，网络可能会生成这样的描述：

‘A’、’ ‘、’c’、‘a’、‘t’、’ ‘、’o’、‘n’、’ ‘、’a’、’ ‘、’b’、‘e’、‘d’

你能描述使用字符级 RNN 的图像描述模型的一个优点吗？你还能描述一个缺点吗？提示：有几个合理的答案，但比较单词级模型和字符级模型参数空间可能会有所帮助。

你的答案：

优点：1. 字符级模型不需要预先定义词汇表，可以处理任何单词，包括训练集中未出现的罕见词、新词或拼写错误。2. 字符集的规模（约 26-52 个字母 + 符号）远小于词汇表规模（数万到数十万），因此字符嵌入矩阵和输出层的参数更少，模型更轻量。3. 字符级模型更容易扩展到多种语言，因为字符集通常较小且通用。

缺点：1. 生成一个单词需要多个时间步（如”cat”需要 3 步），导致序列长度大幅增加，训练和推理时间更长。2. 字符级序列更长，RNN/LSTM 需要学习更长的依赖关系才能正确组装单词，增加了学习难度。3. 模型可能生成不存在的单词或拼写错误的单词（如”ct”而非”cat”），需要后处理或约束解码。